

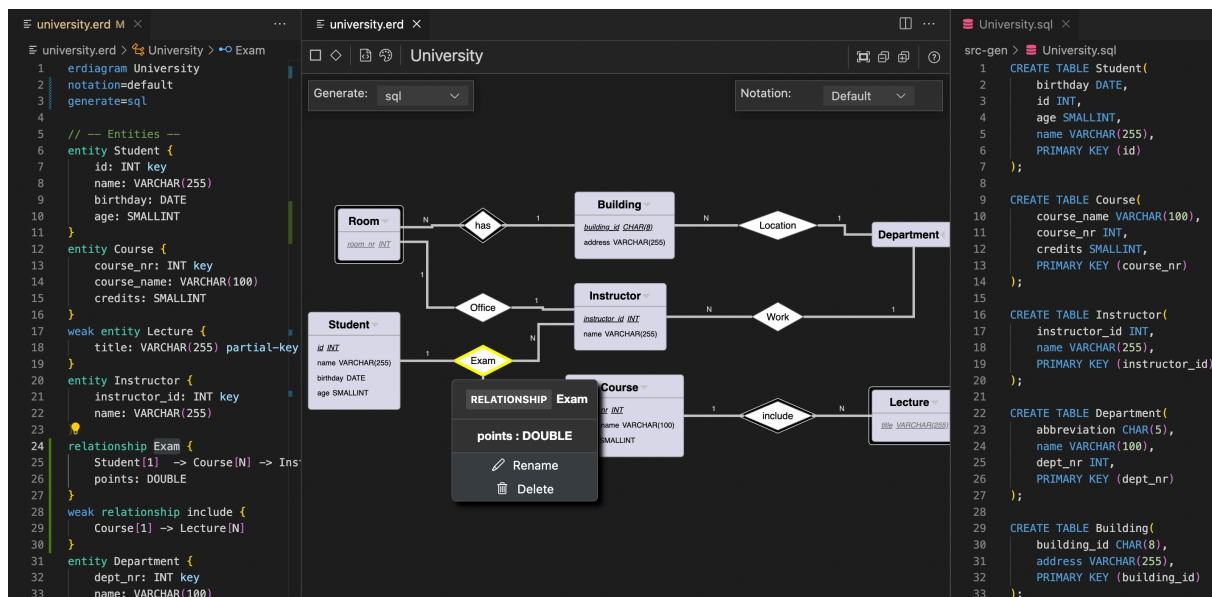
T2: Code Generation: ER Database Schemas

Table of Contents

- [1. Context](#)
 - [2. Motivation](#)
 - [3. Goals](#)
 - [4. Technologies](#)
 - [5. Deliverables](#)
-

1. Context

The projects are situated in the context of **bigER**, an open-source **Entity-Relationship (ER) modeling tool** for **Visual Studio Code (VS Code)**. The central feature of bigER is its **hybrid modeling environment**, in which users can work with a **textual ER language** and a **graphical diagram view** in parallel while both representations remain fully synchronized (i.e., changes in one editor are immediately reflected in the other editor). To realize this behavior, the textual representation is the leading model and graphical edits are first translated into textual changes, which are then propagated back to the diagram. This hybrid design combines the strengths of both forms of modeling, as users can choose the textual editor for fast and efficient textual modeling, while the diagram view offers better usability and a more intuitive overview of the underlying model.

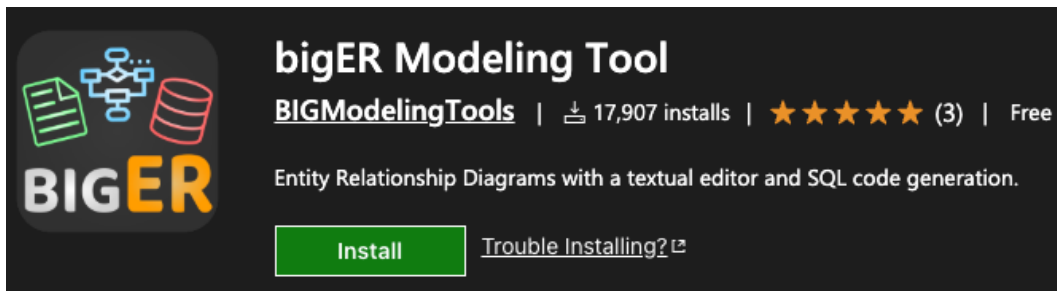


Screenshot of the bigER Modeling Tool

Technically, bigER is realized as a **client-server application**. The original implementation was developed with an **Xtext-based language server** (written in Xtend/Java), combined with **Sprotty** for diagram support and integrated into VS Code as an extension. In this architecture, Xtext provides the grammar-based language infrastructure and default language-server functionality (similar to langium), while Sprotty handles the generation and rendering of graphical models, interactive diagram behavior, and the connection to the VS Code client.

The original bigER implementation already went beyond a minimal editor. It provides a custom ER language with typical language-server features such as validation and rich editor support (via the Language Server Protocol (LSP)), a synchronized diagram view, and code generation for database-related artifacts. Over time, various theses and student projects extended bigER into a broader tool with additional capabilities such as **multiple notation support** and generation support for different target technologies (e.g., different SQL dialects, NoSQL).

Today, the tool has been downloaded more than 17,500 times, which indicates practical interest and relevance beyond an academic prototype. At the same time, an important limitation of the original implementation is its dependency on **Java** due to the Xtext-based server side. In practice, this can complicate installation and maintenance on different machines and operating systems (e.g., different Java versions or additional setup requirements).



bigER in the VS Code Marketplace

The current effort therefore focuses on a **reimplementation of bigER with Langium**. Langium is an open-source language engineering framework with first-class support for the **Language Server Protocol**, written in **TypeScript** and running on **Node.js**. Moving from Xtext to Langium is therefore not just a framework substitution as it means shifting the language-server side from a Java-based technology stack to a fully TypeScript-based one. This removes the Java dependency and aligns the implementation more closely with the surrounding web-based VS Code extension ecosystem.

At the same time, this migration is not a greenfield project (i.e., not starting from scratch). A minimal Langium-based prototype already exists and demonstrates the fundamental feasibility of the new approach, including the hybrid modeling view and selected basic functionality. The task in the individual projects is therefore not to implement a new ER tool from scratch, but to **systematically rebuild and extend important capabilities of bigER within the new architecture**. The original tool serves as a functional reference point, but its solutions should not simply be copied. Instead, the projects are expected to reassess how specific capabilities can be realized in a Langium-based setting, potentially in a cleaner, more robust, more configurable, and better testable way.

This is also what makes the topics academically interesting as they combine language engineering, model-driven development, tool architecture, and practical software engineering in the context of a real hybrid modeling tool. Each topic thus contributes to the broader goal of establishing a Langium-based successor to bigER that remains useful as a practical modeling tool inside VS Code while being easier to evolve, test, and maintain than the original Java-based implementation.

2. Motivation

A central use case of an ER modeling tool is not only to document a schema, but also to derive **deployable artifacts** from it. Users model an ER schema in bigER and expect the tool to generate implementation-oriented outputs such as **SQL DDL** and further persistence-related artifacts. This makes code generation a bridge between conceptual modeling and actual implementation of a system.

At the same time, ER-to-database mapping is not fully deterministic. Several modeling decisions can be translated into relational schemas in different valid ways, depending on design goals, technical constraints, and target platforms. Therefore, a useful generator must not only produce output automatically, but do so in a way that is **predictable, configurable, and dialect-aware**. This becomes relevant when targeting multiple SQL dialects or extending generation towards ORM schemas or NoSQL-related targets.

The topic is therefore motivated by the need to turn the Langium-based bigER tool into a more useful modeling environment. A well-designed generator increases the value of the tool substantially, but it also raises interesting technical questions regarding mapping rules, ambiguity handling, configurability, and output quality.

3. Goals

The goal of this topic is to design and implement a **configurable code generation module** for the Langium-based bigER tool that transforms ER models into database-related target artifacts.

More specifically, the project should address the following goals:

1. ER → SQL DDL generation

- Generate schema elements such as tables, columns, primary keys, foreign keys, unique constraints, and indexes.
- Apply a configurable and consistent naming strategy for generated artifacts (e.g., table/column names, constraint names).
- Handle incomplete ER specifications (e.g. missing datatypes) with sensible defaults and validations. Detect problematic situations such as naming collisions or risky schema decisions and provide warnings when choices produce risky schemas (e.g., nullable FK on mandatory relation).
- Document the implemented ER-to-SQL mapping rules clearly.

2. Ambiguity handling via configuration

- Make important mapping decisions explicitly configurable.
- Relevant examples include inheritance strategies, handling of N:M and 1:1 relations, weak entities, multivalued attributes, and optionality.

3. Dialect support

- Support at least PostgreSQL and MySQL.
- Explicitly address dialect-specific differences such as quoting, type systems, auto-increment strategies, constraints, indexes, and namespace handling.

4. Additional generation targets (optional extension)

- Investigate and potentially support additional targets beyond SQL, such as ORM schemas (e.g., Hibernate, Prisma, SQLAlchemy, etc.), MongoDB, Neo4j, or similar outputs.

5. Testing

- Implement **unit tests** for mapping rules, configuration handling, and dialect-specific generation.
- Include **golden file tests** for representative example models.
- Include **negative tests** for invalid configurations, ambiguous mappings without configuration, or keyword and naming conflicts.

4. Technologies

The following technologies are recommended as a starting point. They are **not mandatory choices**.

- **TypeScript / Node.js:** Core implementation environment of the Langium-based bigER tool.
- **Langium:** Relevant for accessing and processing the parsed ER model, including AST-based generation logic.
- **Existing `export` package code skeleton:** Intended as the main starting point for the implementation and should be extended accordingly.
- **Vitest, Jest, or similar:** Recommended for unit tests.

- **SQL + dialect knowledge:** Especially PostgreSQL and MySQL, including differences in types, identifiers, constraints, and generation syntax.
- **Optional templating or code-generation utilities:** May be useful if you decide to structure the generator output through templates or dedicated emitter abstractions.
- **VS Code Extension API / CLI integration:** For exposing the generator through a command in the extension and through a command-line interface.

5. Deliverables

The expected deliverables for this topic are:

- **Code contribution** in the group's designated repository / branch. This includes:
 - A working **ER → SQL DDL generator**, including configuration support and at least **PostgreSQL** and **MySQL** as target dialects.
 - Integration of the generator into the extension and a CLI (to be decided). Integration into the graphical editor is not required.
 - A suitable **test suite**, including representative unit tests and output-based tests.
- **Documentation** of:
 - the implemented mapping rules,
 - supported configuration options,
 - dialect differences,
 - and the supported feature matrix (which ER concepts are supported).
- A **final report** describing the implemented solution, design decisions, limitations, and recommendations for future work.